Pharma Research Agent

An agentic assistant that answers compound questions about drug labels, with citations and a built-in reviewer

Your Name

GenAI Bootcamp – Coding Assignment

November 2026

The problem

Goal: answer *compound* pharmacology questions from a local corpus of 21 files (FDA DailyMed labels, MedlinePlus monographs, a glossary).

Required behaviour

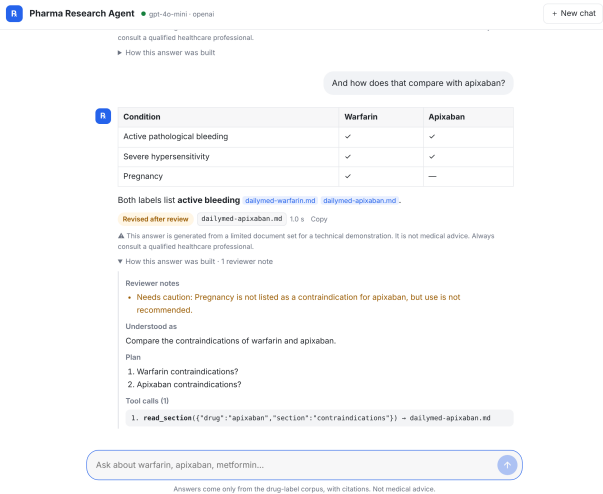
1. **Plan** – split the question into sub-questions
2. **Retrieve** – in-memory similarity search
3. **Tools** – at least two, chosen by the agent
4. **Review** – a critic flags unsupported / ambiguous claims
5. **Answer** – cite source files + medical disclaimer

Example questions

- Main contraindications of warfarin?
- Compare warfarin and apixaban contraindications.
- Warfarin + sertraline: bleeding risk and monitoring?
- eGFR 25 mL/min: is metformin contraindicated?

Delivered as a Docker container: `GET /health`, `POST /ask`, web UI.

The result: a chat that shows its work



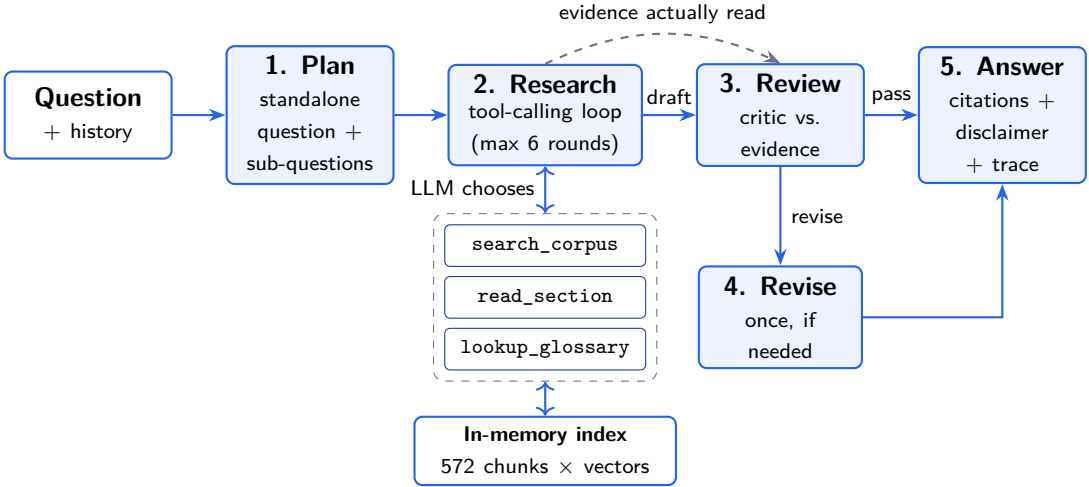
- Answer in Markdown with **inline citations** `[dailymed-warfarin.md]`
- **Verified by reviewer** or **Revised after review** badge
- Reviewer notes, the **plan** and the full **tool trace**
- Follow-up questions in the same chat (*"and for apixaban?"*)
- Disclaimer always present, added by code

Tools and technology choices

Need	Choice	Why
LLM access	OpenAI Python SDK	Works with OpenAI, the LiteLLM proxy and Groq via one client
Orchestration	**own loop** ($\sim$40 lines)	Every step readable, debuggable and returned as a trace; no hidden prompts
Tool use	native function calling	The LLM picks the tool – nothing hard-coded
Vector search	NumPy matrix	572 chunks fit in memory; no vector DB to run
Embeddings	API embeddings / local TF-IDF	Groq has no embeddings API $\rightarrow$ own TF-IDF
API	FastAPI + Pydantic	Validated input/output, typed JSON responses
UI	one HTML file, plain JS	No build step; served by the API itself
Delivery	Docker (`python:3.12-slim`)	Self-contained; graders inject their own `.env`

$\approx$ 860 lines of Python, one file per concern: `corpus.py`, `tools.py`, `agent.py`, `prompts.py`, `llm.py`, `main.py`.

Architecture and workflow



LLM calls per question: **3 minimum** (plan, research, review), **about 10 maximum**.

Step 1 – Planning (and follow-up questions)

- One LLM call in **JSON mode**
- Output: 1–4 self-contained sub-questions + drugs involved
- Plan is injected into the researcher's system prompt
- Robust parsing: falls back to the original question if the JSON is unusable

Conversation for free: the planner also rewrites a follow-up into a *standalone question*. The rest of the pipeline never sees the history ⇒ no extra LLM call.

```
History: "Contraindications of warfarin?"
New: "And for apixaban?"

{ "standalone_question":
    "What are the contraindications
     of apixaban?",
  "sub_questions": [
    "What are apixaban's
     contraindications?"],
  "drugs": ["apixaban"] }
```

Algorithm 1 – Section-aware chunking

Idea: drug labels are organised by sections, so chunk along the Markdown heading tree instead of fixed-size windows.

1. Walk the lines; keep the stack of headings (H1–H6)
2. One chunk per (sub)section, labelled with its path:
   `7 DRUG INTERACTIONS > 7.3 Drugs that Increase Bleeding Risk`
3. Sections longer than 1 500 chars are split on paragraph boundaries
4. Each chunk is embedded *with* its header `[file] title > section`

Why

- Every hit maps to a citable **file + section**
- Passages stay self-contained (no list cut in half)
- The header puts the **drug name** into the vector, so "contraindications" of warfarin and apixaban don't look identical

21 files → **572 chunks**, median ≈ 590 characters.

Algorithm 2 – In-memory similarity search

All chunk vectors are L2-normalised once and stacked in a matrix $V \in \mathbb{R}^{n \times d}$. For a query vector q:

$$\text{score} = V\,\hat{q}, \qquad \hat{q} = \frac{q}{\|q\|}$$

i.e. **cosine similarity** with one matrix-vector product, then top-k ($k = 5$).

Drug filter: `search_corpus(query, drug)` restricts candidates to that drug's files.

Why the filter matters

In "warfarin + sertraline", warfarin's long bleeding sections dominate the ranking. Searching each drug separately guarantees both labels are read.

Startup

The index is built in a **background thread** when the API starts, so `/health` answers 200 immediately.

Algorithm 3 – Local TF-IDF embeddings (no embeddings API)

Groq has no embeddings endpoint, so I wrote a TF-IDF vectoriser in NumPy:

- Tokens: lower-cased words minus stop words, **plus bigrams** (`renal_impairment`)
- Sub-linear term frequency and smoothed IDF:

$$w_{t,c} = (1 + \log \mathrm{tf}_{t,c}) \cdot \left(\log \frac{1+N}{1+\mathrm{df}_t} + 1 \right)$$

- L2-normalised $\Rightarrow$ the *same* cosine search as with API embeddings

Vocabulary learnt from the corpus: **31 457 terms**.

Why it works here

Drug-label questions hinge on **exact terms**: drug names, section names, "eGFR", "INR". Lexical matching is strong on this vocabulary.

Trade-off

No synonyms ("bleeding" $\neq$ "hemorrhage"). Semantic API embeddings are used whenever available (default with OpenAI / LiteLLM).

Step 2 – Agent-driven tool use

Three local tools

`search_corpus` – open questions
`read_section` – *complete* lists
`lookup_glossary` – INR, eGFR…

- Schemas sent as JSON: **the model decides**
- Unknown section ⇒ the tool lists the available ones and the agent retries
- Max 6 rounds, then a forced answer

```python
for _ in range(MAX_TOOL_ROUNDS):
    msg = llm.chat(messages, tools=TOOL_SPECS)
    if not msg.tool_calls: # the model is done
        return msg.content, evidence, steps
    messages.append(assistant_msg(msg))
    for call in msg.tool_calls:
        name = call.function.name
        args = parse_json(call.function.arguments)
        output, chunks = run_tool(index, name, args)
        evidence.extend(chunks) # for the critic
        steps.append(ToolStep(tool=name, ...)) # UI trace
        messages.append({"role": "tool",
                         "tool_call_id": call.id,
                         "content": output})

# budget exhausted: answer with what was read
msg = llm.chat(messages, tools=TOOL_SPECS,
               tool_choice="none")
```

Steps 3–4 – Critic and revision

Critic = a second LLM call with a strict reviewer prompt. It receives:

- the question and the draft answer
- **exactly the passages the agent read** (deduplicated, capped at 40k chars)

It returns JSON:

- `unsupported_claims` – not in the evidence
- `ambiguous_claims` – clinically risky without nuance
- `missing` – parts of the question not answered
- `verdict`: **pass** / **revise** + feedback

Why give it the evidence?

Without it, the critic can only judge plausibility. With it, it can catch claims that came from the model's own memory instead of the corpus.

Revision

If **revise**: **one** rewrite using the feedback and the same evidence. Bounded cost and latency; the notes are shown to the user either way.

Step 5 – Safety by construction, not by prompt

- **Disclaimer added by code** (`config.DISCLAIMER`) – the model cannot forget it
- **Sources = cited ∩ read**: file names are extracted from the answer and kept only if a tool actually returned that file ⇒ an invented citation never reaches the user
- **Answer only from the corpus**: prompts forbid outside knowledge; the critic enforces it
- **Bounded agent**: max 6 tool rounds, 1 revision, evidence cap, SDK retries on rate limits
- **Full transparency**: plan, every tool call and its sources, reviewer verdict and notes, latency – all returned by `POST /ask` and shown in the UI

Response of `POST /ask`

```
question, standalone_question, answer, sources[], plan[], steps[{tool,
arguments, sources}], review{verdict, unsupported_claims, ambiguous_claims,
missing, revised}, disclaimer, latency_ms
```

Testing and evaluation

15 offline unit tests (`pytest`, no network, < 1 s)

- Fake hashed bag-of-words embedding
- **Scripted fake LLM** that returns tool calls, so the whole loop and the API run end to end
- Chunking, tools, TF-IDF ranking, follow-up rewriting, input validation

Evaluation script `python -m app.evaluate`

- Runs the 4 public questions in-process or against the container (`--url`)
- Pass = every expected source file is cited

Question	Sources	Review
Warfarin contra-indications	?/1	?
Warfarin vs apixaban	?/2	?
Warfarin + sertraline	?/2	?
Metformin, eGFR 25	?/1	?

Fill in with the output of `python -m app.evaluate` on the provided key.

Challenges and how I solved them

Challenge	Solution
Similarity search misses items of long lists ("list *all* contraindications")	`read_section` tool returns the whole section; the agent chooses it
In multi-drug questions one drug crowds out the other	`drug` filter on `search_corpus`; prompt: check each drug in its own files
Groq has no embeddings API	Own TF-IDF vectoriser with bigrams, same cosine search
Model may cite files it never read	`sources` = cited $\cap$ actually read
LLM JSON is not always clean	JSON mode + tolerant parser + safe fallbacks
Rate limits on free tiers	SDK retries with back-off; configurable evidence cap
Slow startup (embedding the corpus)	Index built in a background thread; `/health` is instant
Follow-up questions ("and for apixaban?")	Planner rewrites them into standalone questions
Testing an LLM app reliably	Scripted fake LLM + fake embeddings, fully offline

Limits of the current solution

Quality

- Critic is the **same LLM** – it can share the writer's blind spots
- Only **one revision**, and the revised answer is not re-reviewed
- Citations checked at **file level**, not claim by claim
- TF-IDF has no synonyms; brand names (Coumadin, Eliquis) are not mapped to files
- Evaluation only checks sources, not answer correctness

Engineering

- **Latency**: 3–10 sequential LLM calls, no streaming; UI progress labels are timed, not real events
- Index rebuilt (re-embedded) at every start; nothing cached on disk
- In-memory index fits $\sim$20 files, not a large label database
- No authentication or rate limiting; chat history kept only in the browser

What I would improve next

Answer quality

- **Hybrid retrieval**: BM25 + embeddings fused with reciprocal rank fusion, then a cross-encoder re-ranker
- **Claim-level grounding**: split the answer into claims, verify each against its cited passage
- Critic on a different / stronger model; loop until pass (with a budget)
- Drug synonym and brand-name map

Product and operations

- **Streaming** (SSE) of real progress and tokens to the UI
- Parallel tool calls and async LLM calls to cut latency
- Cache embeddings on disk; vector DB if the corpus grows
- Larger eval set with an LLM-as-judge on expected facts
- Tracing of tokens, cost and latency per step

How I worked: AI-assisted development

- Built with **Claude** as a coding assistant; its instructions are committed in `CLAUDE.md`:
  - plain Python, no framework, one file per concern
  - tool choice by the LLM only; disclaimer added by code
  - all LLM access through `app/llm.py`; dependencies pinned
  - tests must run offline with fakes
- Workflow: change code → `pytest -q` → check the API and UI manually
- I reviewed every file and can explain each design decision

Run it

```
docker build -t agent .  && docker run -p 8000:8000 --env-file .env agent
python -m app.evaluate --url http://localhost:8000
```

Summary

- A **transparent agent**: plan → tool-driven research → grounded critic → cited answer
- **Retrieval designed for drug labels**: section-aware chunks, drug filter, full-section reads, glossary
- **Safety in code**: disclaimer, verified sources, bounded loop
- **Provider-agnostic**: OpenAI / LiteLLM with embeddings, or Groq with local TF-IDF
- **Usable**: chat UI with follow-ups, Docker, offline tests, evaluation script

Thank you – questions?

Live demo: `http://localhost:8000`